



FACULTAD DE
ESTUDIOS PROFESIONALES
ZONA MEDIA



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

Práctica 6

Encendido de un LED mediante la Pulsación de un Botón

en Raspberry Pi Pico W

Carrera:

Ingeniería Mecatrónica – Cuarto Semestre

Alumno:

Saúl Martínez Almazán

Docente:

Ing. Luis Jesús Padrón Padrón

Fecha:

25 de Febrero de 2026

Contents

1	Introducción	2
1.1	Objetivo	2
1.2	Importancia del Control de GPIO	2
1.3	Configuración Pull-Down en Pulsadores	2
2	Desarrollo	2
2.1	Configuración del Entorno de Desarrollo	2
2.2	Creación del Proyecto	3
2.3	Proceso de Compilación y Carga	3
3	Resultados	4
3.1	Código Fuente: CMakeLists.txt	4
3.2	Código Fuente: Pulsador.c	4
3.3	Errores Comunes y Soluciones	6
4	Preguntas de Repaso	6
4.1	¿Qué pines del microcontrolador reciben la alimentación externa para no depender del puerto USB?	6
4.2	Explica el funcionamiento de una configuración pull-down en un botón	6
4.3	¿Qué trabajo cumple la función <code>gpio_init()</code> en el código C?	7
4.4	¿Qué trabajo cumple la función <code>gpio_get()</code> en el código C?	7
4.5	¿Qué trabajo cumple la función <code>gpio_put()</code> en el código C?	7
5	Conclusión	7
5.1	Reflexión	7
5.2	Mejoras Propuestas	7
5.3	Aplicación Futura	8
	Anexos	8

1 Introducción

En esta sección se presenta el contexto, objetivo e importancia del presente reporte, el cual documenta el proceso de configuración del entorno de desarrollo para el microcontrolador Raspberry Pi Pico W, así como la implementación de un circuito de control básico que permite encender o apagar un LED en función de la pulsación de un botón mediante el SDK oficial de C/C++.

1.1 Objetivo

Configurar los pines de Entrada/Salida de Propósito General (GPIO) de la Raspberry Pi Pico W para implementar un botón como entrada digital y un LED como salida digital, y aprender a conectar una fuente de alimentación externa al microcontrolador sin depender del puerto USB.

1.2 Importancia del Control de GPIO

El manejo directo de los pines GPIO mediante el SDK de C/C++ permite un acceso de bajo nivel al hardware del microcontrolador RP2040, optimizando el uso de memoria y la velocidad de ejecución. Comprender las entradas y salidas digitales es la base de cualquier sistema embebido de control, desde automatización industrial hasta dispositivos IoT.

1.3 Configuración Pull-Down en Pulsadores

La configuración *pull-down* garantiza un estado lógico definido (cero lógico) cuando el botón no está presionado. Sin esta configuración, el pin de entrada quedaría flotante y podría leer valores aleatorios producidos por ruido electromagnético, causando comportamiento impredecible en el sistema.

2 Desarrollo

2.1 Configuración del Entorno de Desarrollo

La preparación del sistema incluyó la instalación de herramientas esenciales: ARM GCC Toolchain, CMake, Python 3 y Git, además del SDK oficial de Raspberry Pi Pico.

Librerías Utilizadas:

- **pico_stdlib:** Librería estándar que agrupa funciones comunes como el control de GPIO y retardos (`sleep_ms`). Incluye `gpio_init()`, `gpio_set_dir()`, `gpio_get()` y `gpio_put()`.

Estructura del Proyecto:

Se creó la carpeta `Práctica_6` con la siguiente organización:

- `.vscode/` — Configuración del editor
- `pico_sdk_import.cmake` — Enlace al SDK
- `CMakeLists.txt` — Archivo de configuración del sistema de compilación
- `Pulsador.c` — Código fuente principal

2.2 Creación del Proyecto

Terminal de Desarrollador:

Se utilizó el *Pico – Developer PowerShell*, el cual preconfigura las rutas del SDK (`PICO_SDK_PATH`) automáticamente.

Archivo `CMakeLists.txt`:

Este archivo es el núcleo del sistema de compilación. Debe:

1. Declarar la versión de CMake requerida y el tipo de placa (`pico_w`)
2. Incluir el archivo `pico_sdk_import.cmake`
3. Definir el ejecutable con el archivo `Pulsador.c`
4. Vincular la librería `pico_stdlib`

Código en C:

Se implementó una rutina de bucle infinito que lee el estado del pin de entrada (botón) y en función de dicho estado controla el nivel lógico del pin de salida (LED), con un mecanismo de *debounce* por software de 50 ms.

2.3 Proceso de Compilación y Carga

1. **Compilación en VS Code:** Se ejecutó el comando `cmake ..` dentro de la carpeta `build` para generar los archivos de construcción, seguido de `nmake` o `make` para compilar el binario.
2. **Generación del `.uf2`:** El proceso finaliza creando un archivo con extensión `.uf2`, listo para ser cargado al microcontrolador.
3. **Transferencia:** Se conectó el microcontrolador manteniendo presionado el botón `BOOTSEL`, montándolo como unidad de almacenamiento masivo (`RPI-RP2`). El archivo `.uf2` se arrastró a dicha unidad.
4. **Verificación:** El programa se ejecuta inmediatamente tras la carga. Al presionar el botón, el LED enciende; al soltarlo, apaga.

3 Resultados

3.1 Código Fuente: CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.13)
2
3 include(pico_sdk_import.cmake)
4 set(PICO_BOARD pico_w)           # Le decimos al compilador que
   la placa es la W
5
6 project(Practica_6)              # El nombre de la carpeta de
   proyecto
7
8 pico_sdk_init()                  # Inicializa el SDK del proyecto
9
10 add_executable(Pulsador          # Nombre del archivo sin
   terminacion .c
11     Pulsador.c                  # Nombre del archivo de código
   con extension .c
12 )
13
14 target_link_libraries(Pulsador   # Nombre del archivo sin
   terminacion .c
15     pico_stdlib                  # Librerías que se estarán
   usando en el proyecto
16 )
17
18 pico_add_extra_outputs(Pulsador) # Módulo que genera versiones
   del código compilado
```

3.2 Código Fuente: Pulsador.c

A continuación se presenta el código fuente en C desarrollado para el control del LED mediante pulsador en la Raspberry Pi Pico W:

```
1 #include "pico/stdlib.h" // Biblioteca para GPIO y
   temporización
2
3 int main() {
4     const uint LED_PIN = 3; // Define el pin donde se
   conecta el LED
5     const uint BOTON = 4; // Define el pin donde se
   conecta el boton
6
7     gpio_init(LED_PIN); // Inicializa el pin del
   LED
```

```
8      gpio_init(BOTON);                // Inicializa el pin del
      boton
9      gpio_set_dir(LED_PIN, GPIO_OUT); // Configura el pin del
      LED como salida
10     gpio_set_dir(BOTON, GPIO_IN);    // Configura el pin del
      boton como entrada
11
12     while (true) { // Bucle infinito para mantener el
      programa en ejecucion
13         if (gpio_get(BOTON)) { // Si el boton esta presionado
      (devuelve 1)
14             sleep_ms(50); // Pausa 50ms para evitar rebotes
      mecanicos (debounce)
15             if (gpio_get(BOTON)) { // Vuelve a verificar si
      sigue presionado
16                 gpio_put(LED_PIN, 1); // Enciende el LED (3.3
      V en el pin)
17             }
18         } else {
19             gpio_put(LED_PIN, 0); // Apaga el LED (0V) si el
      boton no esta presionado
20         }
21         sleep_ms(10); // Pequeno delay para evitar sobrecarga
      en el microcontrolador
22     }
23 }
```

3.3 Errores Comunes y Soluciones

Table 1: Errores encontrados y soluciones aplicadas durante el desarrollo

Error Encontrado	Solución Aplicada
El LED no responde al presionar el botón.	Se verificó que el pin del botón estuviera configurado como entrada (GPIO_IN) y que la resistencia pull-down de 10 k Ω estuviera correctamente conectada a GND.
El LED parpadea erráticamente sin presionar el botón.	Se implementó el mecanismo de <i>debounce</i> por software mediante <code>sleep_ms(50)</code> y doble lectura del pin para filtrar rebotes mecánicos.
El código no compila por falta de PICO_SDK_PATH.	Se configuró correctamente la variable de entorno o se utilizó el <i>Developer PowerShell</i> que la preconfigura automáticamente.

4 Preguntas de Repaso

4.1 ¿Qué pines del microcontrolador reciben la alimentación externa para no depender del puerto USB?

Según el *datasheet* de la Raspberry Pi Pico W (apartado 3.5, *Powering Raspberry Pi Pico W*), la alimentación externa se suministra a través del pin **VSYS** (pin físico 39), que acepta un rango de tensión de 1.8 V a 5.5 V. Un regulador conmutado interno genera los 3.3 V necesarios para el RP2040 y sus periféricos. El retorno de corriente se realiza por cualquiera de los pines GND. También es posible alimentar directamente en 3.3 V a través del pin **3V3** (pin 36), desactivando previamente el regulador interno para evitar conflictos.

4.2 Explica el funcionamiento de una configuración pull-down en un botón

En una configuración *pull-down*, el pin de entrada se conecta a GND (0 V) a través de una resistencia de valor alto (10 k Ω). Cuando el botón está abierto, la resistencia mantiene el pin a cero lógico (0 V), evitando señales flotantes. Al presionar el botón, el pin se conecta directamente a 3.3 V y el microcontrolador lee un uno lógico. La resistencia limita la corriente hacia GND y protege la fuente de alimentación de cortocircuitos.

4.3 ¿Qué trabajo cumple la función `gpio_init()` en el código C?

La función `gpio_init(uint gpio)` inicializa el pin GPIO indicado en un estado base conocido: establece su función como GPIO de propósito general, habilita el módulo de E/S, lo deja en dirección de entrada por defecto y desactiva las resistencias *pull-up* y *pull-down* internas. Es el primer paso obligatorio antes de `gpio_set_dir()` y antes de leer o escribir en el pin.

4.4 ¿Qué trabajo cumple la función `gpio_get()` en el código C?

La función `gpio_get(uint gpio)` lee el estado lógico actual del pin especificado. Retorna `true` (1) si detecta un nivel alto (≈ 3.3 V) o `false` (0) si detecta un nivel bajo (≈ 0 V). En esta práctica se utiliza para determinar si el botón está presionado: al presionarlo en configuración *pull-down* la función retorna 1, activando la rama que enciende el LED.

4.5 ¿Qué trabajo cumple la función `gpio_put()` en el código C?

La función `gpio_put(uint gpio, bool value)` establece el nivel lógico de salida del pin indicado. Con `value = 1` el pin se pone en alto (3.3 V), encendiendo el LED. Con `value = 0` el pin se pone en bajo (0 V), apagándolo. Esta función solo tiene efecto si el pin fue previamente configurado como salida mediante `gpio_set_dir()`.

5 Conclusión

5.1 Reflexión

La implementación de este circuito de control por pulsador permitió afianzar los conceptos fundamentales del manejo de pines GPIO en la Raspberry Pi Pico W. A diferencia de entornos interpretados como MicroPython, el SDK de C/C++ ofrece un control preciso sobre el hardware, lo que resulta indispensable en el desarrollo de sistemas embebidos robustos y de tiempo real.

La configuración *pull-down* y el mecanismo de *debounce* por software son técnicas esenciales en cualquier diseño electrónico que incorpore pulsadores, ya que garantizan lecturas confiables y eliminan artefactos causados por la naturaleza mecánica de los interruptores.

5.2 Mejoras Propuestas

Se podría implementar el uso de *interrupciones por hardware* en lugar del sondeo (*polling*) del pin, lo cual presenta las siguientes ventajas:

- **Eficiencia:** El procesador solo actúa cuando ocurre el evento, liberando ciclos de reloj para otras tareas.
- **Multitarea:** Permite ejecutar otras rutinas mientras espera la pulsación del botón.

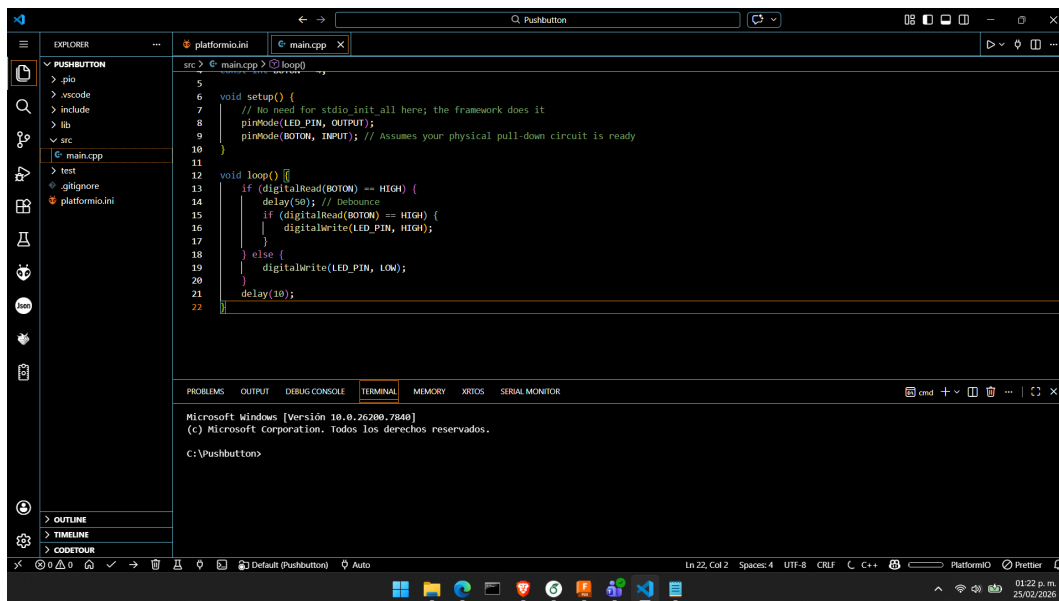
- **Eficiencia energética:** El procesador puede entrar en modos de bajo consumo entre eventos.

5.3 Aplicación Futura

El manejo de entradas y salidas digitales es la base de proyectos más complejos, como sistemas de alarma, control de actuadores, lectores de sensores digitales e interfaces hombre-máquina (HMI). Este flujo de trabajo sienta las bases para el desarrollo de sistemas embebidos autónomos alimentados por batería en aplicaciones de IoT y automatización.

Anexos

Anexo A – Evidencia de Compilación en Terminal



```
5
6 void setup() {
7   // No need for stdio init all here; the framework does it
8   pinMode(LED_PIN, OUTPUT);
9   pinMode(BOTON, INPUT); // Assumes your physical pull-down circuit is ready
10 }
11
12 void loop() {
13   if (digitalRead(BOTON) == HIGH) {
14     delay(50); // Debounce
15     if (digitalRead(BOTON) == HIGH) {
16       digitalWrite(LED_PIN, HIGH);
17     }
18   } else {
19     digitalWrite(LED_PIN, LOW);
20   }
21   delay(10);
22 }
```

Microsoft Windows [Versión 10.0.26200.7848]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Pushbutton>

Figure 1: Evidencia de Terminal

Anexo B – Armado en Protoboard

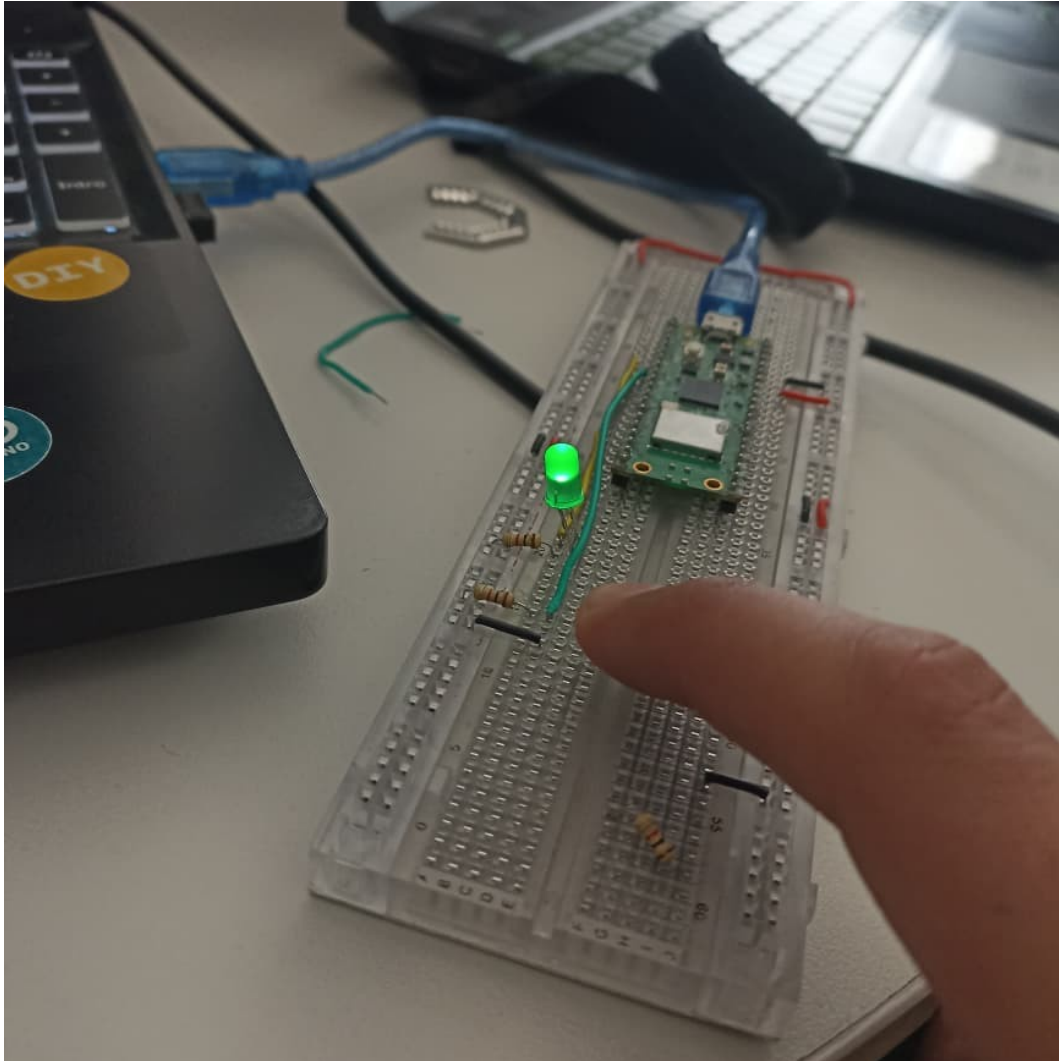


Figure 2: Diagrama en protoboard del circuito

References

- [1] Raspberry Pi Ltd., *Raspberry Pi Pico C/C++ SDK* (v2.0). <https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>, 2024.
- [2] Raspberry Pi Ltd., *Raspberry Pi Pico W Datasheet*. <https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>, 2024.
- [3] Raspberry Pi Ltd., *RP2040 Datasheet*. <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>, 2024.